NATIONAL JUDICIAL CASEMANAGEMENT SYSTEM

Project Report

Case Study 189: National Judicial Case Management System

Name:-Pratik swain
Roll Number:-150096725184
College:-ITM Skills University,Kharghar
Batch:-2025-2029
Course:BTech (CSE)
Subject:-DSA-1

Project Title

National Judicial Case Management System

A console-based software solution developed using C++ and Data Structures to efficiently manage judicial case records, prioritize urgent matters, analyze legal precedents, track appeal journeys, and detect circular legal citations.

Problem Statement

Case Study 189: National Judicial Case Management System

Introduction

Judicial systems handle millions of legal cases annually. Managing case registrations, prioritizing
urgent matters, scheduling hearings, and retrieving records efficiently are critical for delivering timely justice. The National Judicial Case Management System is designed to streamline court operations through intelligent case management and legal analytics.

Objective
The objective is to manage nationwide case records, prioritize urgent matters, automate scheduling, and provide instant access to legal information.

Industry Context
Courts process diverse case types, including criminal, civil, constitutional, and commercial disputes. Efficient case management improves transparency and reduces delays.

Case Registration Using Queue
Newly filed cases enter a Queue and are processed in filing order, ensuring fairness and transparency.

Urgent Matter Handling Using Priority Queue
Emergency cases, bail applications, and constitutional matters are prioritized using a Priority Queue.

Case Retrieval Using Binary Search
Case records are stored in sorted databases, enabling rapid retrieval through Binary Search.
Legal Precedent Analysis Using Graphs
Judicial citations form a graph structure where cases are nodes and citations are edges. Graph analysis identifies influential judgments and precedent relationships.

Appeal Tracking Using DFS
DFS traces appeal histories from lower courts to higher courts, providing complete legal case Journeys.

Deliverables
The system provides digital court dashboards, automated hearing scheduling, legal analytics, precedent discovery, appeal tracking, and case search portals.

Conclusion
The National Judicial Case Management System demonstrates how Queue, Priority Queue, Binary Search, Graphs, and DFS can modernize judicial administration and improve access to justice.

Objectives

The primary objectives of the project are:

1. To automate judicial case registration and storage.
2. To provide fast case retrieval using Binary Search.
3. To process filed cases using Queue data structures.
4. To prioritize urgent cases using a Max Heap.
5. To model legal citations using Graphs.
6. To identify influential judgments through citation analysis.
7. To track appeals using Depth First Search (DFS).
8. To detect circular legal references using Graph Cycle Detection.
9. To demonstrate practical applications of Data Structures and Algorithms.

Design / Architecture

System Overview

The National Judicial Case Management System is designed as a modular, menu-driven application that simulates the workflow of a real judicial administration system. The architecture integrates multiple Data Structures and Algorithms to efficiently manage case records, legal precedents, appeal hierarchies, and priority-based scheduling.

The system follows a layered architecture where user requests are processed through specialized modules, each responsible for a specific judicial operation.

Architectural Components

1. User Interaction Layer

The User Interaction Layer acts as the interface between court administrators and the system. It provides menu-driven operations through which users can:

- Register new cases
- Search existing cases
- Process pending filings
- Handle urgent matters
- Analyze legal precedents
- Track appeal journeys
- Generate system statistics

All user inputs are validated before being forwarded to the corresponding processing modules.

---

2. Case Management Module

This module is responsible for maintaining all judicial case records.

Each case contains:

- Case ID
- Case Title
- Case Type
- Priority Level
- Current Status
- Hearing Date

Cases are stored inside a centralized array repository which serves as the primary database of the system.

Responsibilities

- Register new cases
- Update case information
- Store case metadata

- Support searching and reporting operations

Data Structure Used

Array

Reason for Selection

Arrays provide direct indexing and efficient storage for a moderate number of case records while keeping implementation simple and suitable for educational DSA projects.

---

3. Filing Queue Module

When a new case is registered, it enters the filing queue before being processed.

The queue follows the First-In-First-Out (FIFO) principle.

Workflow
Case A Registered
Case B Registered
Case C Registered

Processing Order:
A → B → C

Responsibilities

- Maintain filing order
- Ensure fairness in processing
- Handle pending registrations

Data Structure Used

Queue

Reason for Selection

Court filings are naturally processed in the order they are received. Queue provides an efficient and realistic representation of this workflow.

---

4. Urgent Case Management Module

Certain judicial matters require immediate attention due to their critical nature.

Examples:

- Murder trials
- Constitutional petitions
- Emergency injunctions
- Bail applications

Such cases are assigned higher priority scores and stored in a Max Heap.

Workflow
Priority 10 → Case 105
Priority 9 → Case 103
Priority 8 → Case 114

The heap always places the highest-priority case at the root.

Responsibilities

- Prioritize urgent matters
- Retrieve highest-priority case quickly
- Improve judicial response time

Data Structure Used

Max Heap

Reason for Selection

Max Heap allows efficient retrieval of the most urgent case in O(log n) time.

---

5. Search and Retrieval Module

Judicial systems frequently require retrieval of case records.

To improve search efficiency, cases are sorted according to their Case IDs and searched using Binary Search.

Workflow
Sorted IDs:

101 102 103 104 105 106

Search 105

Step 1:
Mid = 103

Step 2:
Move Right

Step 3:
Found 105

Responsibilities

- Fast case retrieval
- Reduce search time
- Improve user productivity

Algorithm Used

Binary Search

---

6. Legal Precedent Analysis Module

Judicial decisions often reference previous judgments.

These relationships are represented using a Citation Graph.

Example
104 → 103
104 → 101
104 → 102

Meaning:

Case 104 references Cases 103, 101, and 102.

Responsibilities

- Store precedent relationships
- Analyze landmark judgments
- Determine influence of judgments

Data Structure Used

Directed Graph

Reason for Selection

Graphs naturally model legal citation relationships where one judgment references another.

---

7. Appeal Tracking Module

Legal cases often move through multiple levels of courts.

Example:

District Court
↓
High Court
↓
Supreme Court

This progression is represented using an Appeal Graph.

Example
101 → 201 → 301 → 401

Responsibilities

- Track case movement
- Visualize appeal hierarchy
- Support judicial analytics

Data Structure Used

Directed Graph

Traversal Technique

Depth First Search (DFS)

---

8. Circular Citation Detection Module

In legal systems, circular references between judgments may create inconsistencies.

Example:

120 → 121
121 → 122
122 → 120

This forms a cycle.

Responsibilities

- Detect invalid citation loops
- Improve legal consistency
- Support judicial review

Algorithm Used

DFS with Three-Color Marking

Node States:

- White (0) = Unvisited
- Gray (1) = Visiting
- Black (2) = Visited

When DFS encounters a Gray node again, a cycle exists.

---

9. Analytics and Dashboard Module

This module provides system-wide statistics.

Metrics include:

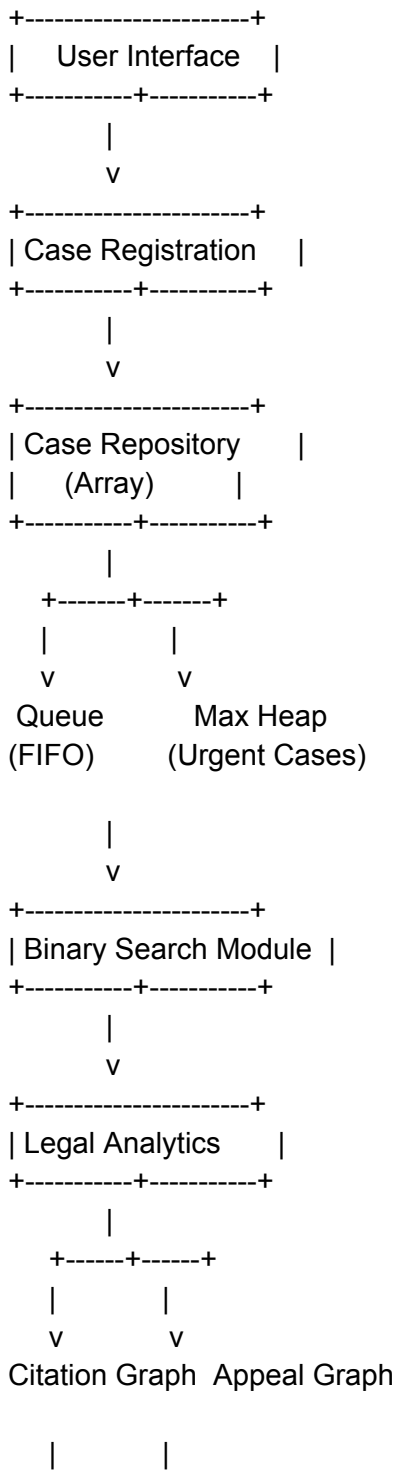
- Total cases
- Pending cases
- Urgent cases
- Citation counts
- Appeal hierarchy counts

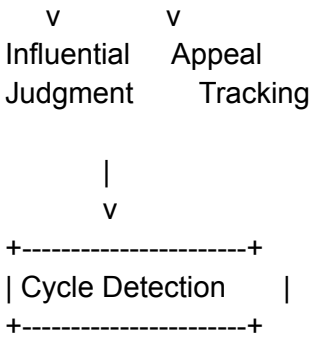
Purpose

- Judicial workload monitoring
- Performance evaluation

- Decision support

Complete System Architecture Diagram





Algorithm Description

Algorithm 1: Case Registration

Objective

Add a new judicial case into the system.

Steps

1. Read case details.
2. Create a Case object.
3. Store case in array repository.
4. Insert case into filing queue.
5. Check priority level.
6. If priority is high, insert into Max Heap.
7. Display success message.

Pseudocode

```
INPUT Case

Store in Array

Enqueue Case

IF Priority >= Threshold
   Insert into Heap
END IF
```

Complexity

Time Complexity: O(1)

Space Complexity: O(1)

Algorithm 2: Queue Processing

Objective

Process cases according to filing order.

Steps

1. Check whether queue is empty.
2. Remove front case.
3. Display processed case.
4. Update queue position.

Pseudocode

```
IF Queue Empty
   Stop
ELSE
   Remove Front Element
END IF
```

Complexity

Time Complexity: O(1)

Space Complexity: O(1)

Algorithm 3: Urgent Case Handling

Objective

Retrieve highest-priority case.

Steps

1. Access root of Max Heap.
2. Remove root.
3. Reorganize heap.
4. Return highest-priority case.

Pseudocode

Remove Heap Root

Heapify

Return Highest Priority Case

Complexity

Time Complexity: O(log U)

Space Complexity: O(1)

Algorithm 4: Binary Search

Objective

Find a case efficiently using Case ID.

Steps

1. Sort cases.
2. Select middle element.
3. Compare target with middle.
4. Move left or right.
5. Repeat until found.

Complexity

Time Complexity: O(log N)

Space Complexity: O(1)

Algorithm 5: Landmark Judgment Analysis

Objective

Find the most influential judgment.

Steps

1. Traverse citation graph.
2. Count incoming references.
3. Store citation frequencies.
4. Identify maximum count.
5. Return judgment with highest citations.

Complexity

Time Complexity: O(V + E)

Space Complexity: O(V)

Algorithm 6: Appeal Journey Tracking

Objective

Trace appeals through court hierarchy.

Steps

1. Start from selected case.
2. Mark node as visited.
3. Visit connected appeal nodes.
4. Continue recursively.
5. Print traversal path.

DFS Traversal Example

101
↓
201
↓
301
↓

401

Complexity

Time Complexity: O(V + E)

Space Complexity: O(V)

Algorithm 7: Circular Citation Detection

Objective

Detect cycles in legal citation network.

Steps

1. Mark node as Visiting.
2. Traverse adjacent nodes.
3. If Visiting node is encountered again:
    - Cycle exists.
4. Otherwise mark node as Visited.
5. Continue DFS.

Example

120 → 121
121 → 122
122 → 120

Cycle Detected.

Complexity

Time Complexity: O(V + E)

Space Complexity: O(V)

Complexity Analysis

Operation	Algorithm / Data Structure	Time Complexity	Space Complexity
Register Case	Array Insert + Queue Enqueue	**O(1)**	**O(1)**
Process Filing Queue	Queue Dequeue	**O(1)**	**O(1)**
Handle Urgent Case	Max Heap Insert/Delete	**O(log N)**	**O(1)**
Search Case	Bubble Sort + Binary Search	**$O(N^2 + \log N) \approx O(N^2)$**	**O(1)**
View All Cases	Bubble Sort	**$O(N^2)$**	**O(1)**
Add Citation Relationship	Graph Edge Insertion	**O(1)**	**O(1)**
Most Influential Judgment Analysis	In-Degree Counting on Graph	**O(V + E)**	**O(V)**
Appeal Journey Tracking	Depth First Search (DFS)	**O(V + E)**	**O(V)**
Circular Citation Detection	DFS Cycle Detection	**O(V + E)**	**O(V)**
Load Sample Dataset	Array Traversal + Graph Construction	**O(N + E)**	**O(1)**

Where:

- **N** = Total number of cases stored in the system
- **V** = Number of vertices (cases) in the graph
- **E** = Number of edges (citations or appeal links)

Screenshots of Execution

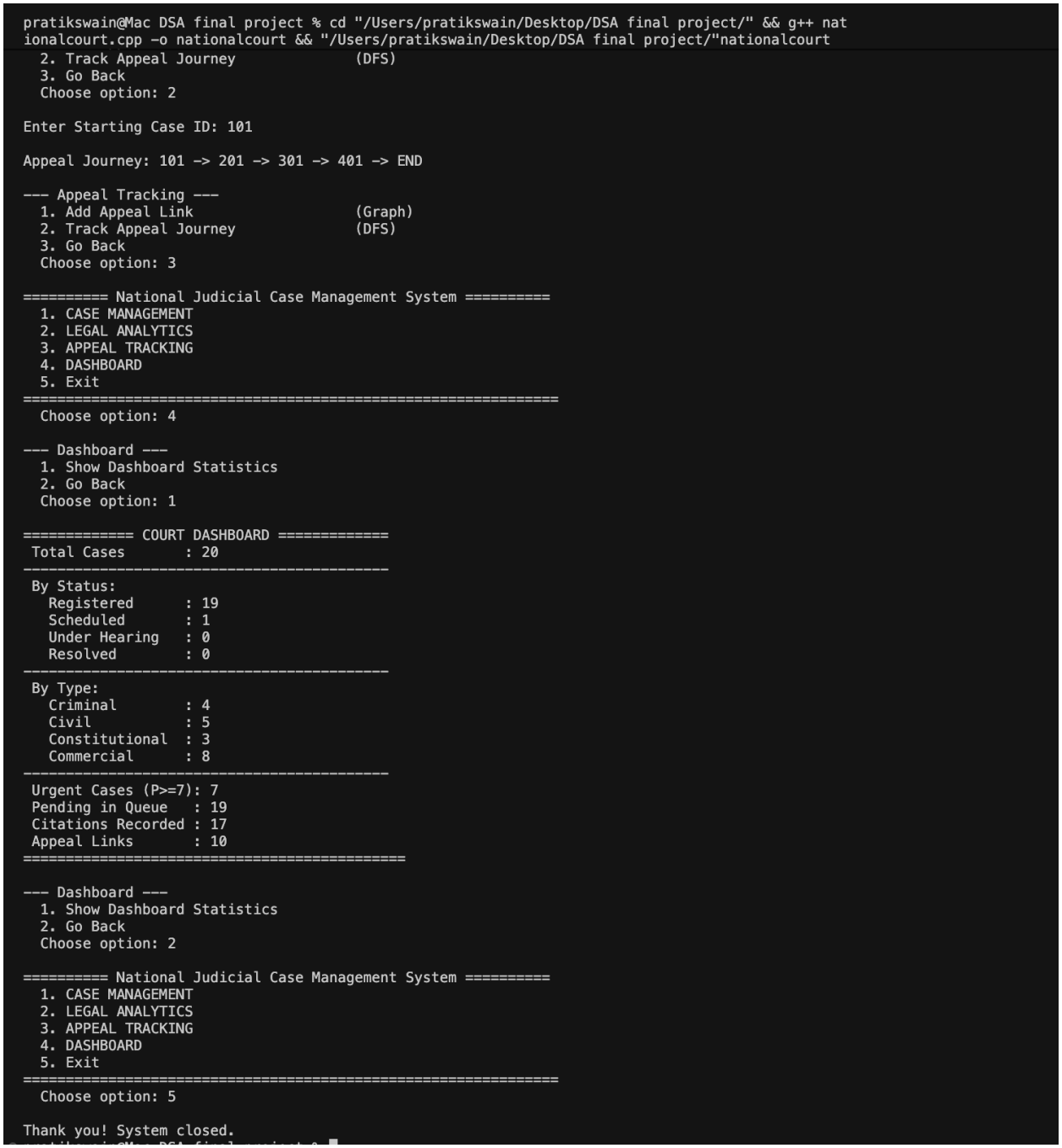
```
pratikswain@Mac DSA final project % clear
pratikswain@Mac DSA final project % cd "/Users/pratikswain/Desktop/DSA final project/" && g++ nat
ionalcourt.cpp -o nationalcourt && "/Users/pratikswain/Desktop/DSA final project/"nationalcourt
[Comprehensive dataset loaded: 19 cases, 12 citation links (including a circular loop), and 3 appeal hierarchies]

================ COURT PORTAL LOGIN =================
Enter Username: admin
Enter Password: admin123

Login Successful! Welcome to the Judicial Portal.


========== National Judicial Case Management System ==========
  1. CASE MANAGEMENT
  2. LEGAL ANALYTICS
  3. APPEAL TRACKING
  4. DASHBOARD
  5. Exit
============================================================
  Choose option: 1

--- Case Management ---
  1. Register New Case              (Queue)
  2. Process Filing Queue            (Queue - FIFO)
  3. Handle Urgent Case              (Priority Queue)
  4. Search Case by ID               (Binary Search)
  5. View All Cases                  (Sorted by Priority)
  6. Filter Cases by Type
  7. Update Case Status
  8. Schedule Hearing
  9. Go Back
  Choose option: 1

--- Register New Case ---
Enter Case ID: 130
Enter Title: roberty of house
Enter Type (Criminal/Civil/Constitutional/Commercial): civil
Enter Priority (1-10): 6
Case 130 registered successfully!

--- Case Management ---
  1. Register New Case              (Queue)
  2. Process Filing Queue            (Queue - FIFO)
  3. Handle Urgent Case              (Priority Queue)
  4. Search Case by ID               (Binary Search)
  5. View All Cases                  (Sorted by Priority)
  6. Filter Cases by Type
  7. Update Case Status
  8. Schedule Hearing
  9. Go Back
  Choose option: 2

--- Processing Case (FIFO Order) ---
ID: 101 | Title: State vs. Sharma (Theft Case) | Type: Criminal | Priority: 4

--- Case Management ---
  1. Register New Case              (Queue)
  2. Process Filing Queue            (Queue - FIFO)
  3. Handle Urgent Case              (Priority Queue)
  4. Search Case by ID               (Binary Search)
  5. View All Cases                  (Sorted by Priority)
  6. Filter Cases by Type
  7. Update Case Status
  8. Schedule Hearing
  9. Go Back
  Choose option: 3

--- Highest Priority Case ---
ID: 110 | Title: Homicide Case: State vs. Vikram | Priority: 10 | Status: Registered
```

```
pratikswain@Mac DSA final project % cd "/Users/pratikswain/Desktop/DSA final project/" && g++ nat
ionalcourt.cpp -o nationalcourt && "/Users/pratikswain/Desktop/DSA final project/"nationalcourt
--- Case Management ---
  1. Register New Case              (Queue)
  2. Process Filing Queue            (Queue - FIFO)
  3. Handle Urgent Case              (Priority Queue)
  4. Search Case by ID               (Binary Search)
  5. View All Cases                  (Sorted by Priority)
  6. Filter Cases by Type
  7. Update Case Status
  8. Schedule Hearing
  9. Go Back
  Choose option: 4

Enter Case ID to search: 101

--- Case Found ---
ID       : 101
Title    : State vs. Sharma (Theft Case)
Type     : Criminal
Priority : 4
Status   : Registered
Hearing  : Not Scheduled

--- Case Management ---
  1. Register New Case              (Queue)
  2. Process Filing Queue            (Queue - FIFO)
  3. Handle Urgent Case              (Priority Queue)
  4. Search Case by ID               (Binary Search)
  5. View All Cases                  (Sorted by Priority)
  6. Filter Cases by Type
  7. Update Case Status
  8. Schedule Hearing
  9. Go Back
  Choose option: 5

--- All Cases (Sorted by Priority: High to Low) ---
----------------------------------------------------------------------
ID: 105 | Murder Trial: State vs. Rao | Type: Criminal | Priority: 10 | Status: Registered
ID: 110 | Homicide Case: State vs. Vikram | Type: Criminal | Priority: 10 | Status: Registered
ID: 103 | Bail Application: State vs. Khan | Type: Constitutional | Priority: 9 | Status: Registered
ID: 113 | Writ Petition: Environmental Pollution Action | Type: Constitutional | Priority: 9 | Status: Registered
ID: 107 | Fundamental Rights Petition (Freedom of Speech) | Type: Constitutional | Priority: 8 | Status: Registered
ID: 114 | Intellectual Property: TechSoft vs. AppCorp | Type: Commercial | Priority: 8 | Status: Registered
ID: 109 | Corporate Tax Evasion: Alpha Ltd | Type: Commercial | Priority: 7 | Status: Registered
ID: 108 | Contract Breach: TechCo vs. DataInc | Type: Commercial | Priority: 6 | Status: Registered
ID: 116 | Bankruptcy Filing: RetailGlobal Group | Type: Commercial | Priority: 6 | Status: Registered
ID: 120 | Patent Dispute: Inventions Inc vs. Creators Ltd | Type: Commercial | Priority: 6 | Status: Registered
ID: 122 | Trademark Opposition: BrandA vs. BrandB | Type: Commercial | Priority: 6 | Status: Registered
ID: 130 | roberty of house | Type: civil | Priority: 6 | Status: Registered
ID: 104 | Trade Fraud: ABC vs. XYZ Corp | Type: Commercial | Priority: 5 | Status: Registered
ID: 115 | Assault & Battery: State vs. Verma | Type: Criminal | Priority: 5 | Status: Registered
ID: 121 | Copyright Infringement: MediaHouse vs. NetBroadcasting | Type: Commercial | Priority: 5 | Status: Registered
ID: 101 | State vs. Sharma (Theft Case) | Type: Criminal | Priority: 4 | Status: Registered
ID: 112 | Defamation Suit: Kapoor vs. Puri | Type: Civil | Priority: 4 | Status: Registered
ID: 102 | Land Boundary Dispute: Patel vs. Gupta | Type: Civil | Priority: 3 | Status: Registered
ID: 111 | Divorce & Custody: Sen vs. Sen | Type: Civil | Priority: 3 | Status: Registered
ID: 106 | Property Rights: Desai vs. Mehta | Type: Civil | Priority: 2 | Status: Registered
----------------------------------------------------------------------
Total Cases: 20

--- Case Management ---
  1. Register New Case              (Queue)
  2. Process Filing Queue            (Queue - FIFO)
  3. Handle Urgent Case              (Priority Queue)
  4. Search Case by ID               (Binary Search)
  5. View All Cases                  (Sorted by Priority)
  6. Filter Cases by Type
  7. Update Case Status
```

```
pratikswain@Mac DSA final project % cd "/Users/pratikswain/Desktop/DSA final project/" && g++ n
ionalcourt.cpp -o nationalcourt && "/Users/pratikswain/Desktop/DSA final project/"nationalcourt
--- Case Management ---
  1. Register New Case              (Queue)
  2. Process Filing Queue            (Queue - FIFO)
  3. Handle Urgent Case              (Priority Queue)
  4. Search Case by ID               (Binary Search)
  5. View All Cases                  (Sorted by Priority)
  6. Filter Cases by Type
  7. Update Case Status
  8. Schedule Hearing
  9. Go Back
  Choose option: 6

Enter Type to filter (Criminal/Civil/Constitutional/Commercial): criminal

--- Cases of Type: criminal ---
ID: 101 | State vs. Sharma (Theft Case) | Priority: 4 | Status: Registered
ID: 105 | Murder Trial: State vs. Rao | Priority: 10 | Status: Registered
ID: 110 | Homicide Case: State vs. Vikram | Priority: 10 | Status: Registered
ID: 115 | Assault & Battery: State vs. Verma | Priority: 5 | Status: Registered

--- Case Management ---
  1. Register New Case              (Queue)
  2. Process Filing Queue            (Queue - FIFO)
  3. Handle Urgent Case              (Priority Queue)
  4. Search Case by ID               (Binary Search)
  5. View All Cases                  (Sorted by Priority)
  6. Filter Cases by Type
  7. Update Case Status
  8. Schedule Hearing
  9. Go Back
  Choose option: 7

Enter Case ID: 101
Current Status: Registered
Select New Status:
  1. Registered
  2. Scheduled
  3. Under Hearing
  4. Resolved
Choice: 2
Status updated to "Scheduled".

--- Case Management ---
  1. Register New Case              (Queue)
  2. Process Filing Queue            (Queue - FIFO)
  3. Handle Urgent Case              (Priority Queue)
  4. Search Case by ID               (Binary Search)
  5. View All Cases                  (Sorted by Priority)
  6. Filter Cases by Type
  7. Update Case Status
  8. Schedule Hearing
  9. Go Back
  Choose option: 8

Enter Case ID: 101
Enter Hearing Date (e.g. 2026-07-15): 2026-06-18
Hearing scheduled for Case 101 on 2026-06-18

--- Case Management ---
  1. Register New Case              (Queue)
  2. Process Filing Queue            (Queue - FIFO)
  3. Handle Urgent Case              (Priority Queue)
  4. Search Case by ID               (Binary Search)
  5. View All Cases                  (Sorted by Priority)
  6. Filter Cases by Type
  7. Update Case Status
  8. Schedule Hearing
```

```
--- Case Management ---
  1. Register New Case              (Queue)
  2. Process Filing Queue            (Queue - FIFO)
  3. Handle Urgent Case              (Priority Queue)
  4. Search Case by ID               (Binary Search)
  5. View All Cases                  (Sorted by Priority)
  6. Filter Cases by Type
  7. Update Case Status
  8. Schedule Hearing
  9. Go Back
  Choose option: 9

========== National Judicial Case Management System ==========
  1. CASE MANAGEMENT
  2. LEGAL ANALYTICS
  3. APPEAL TRACKING
  4. DASHBOARD
  5. Exit
============================================================
  Choose option: 2

--- Legal Analytics ---
  1. Add Legal Citation               (Graph)
  2. Show Precedent Graph             (Graph)
  3. Most Influential Judgment         (Graph Analysis)
  4. Detect Circular Citations         (Cycle Detection)
  5. Go Back
  Choose option: 1

Enter Case ID: 110
Enter Cited Case ID: 130
Citation added: Case 110 cites Case 130

--- Legal Analytics ---
  1. Add Legal Citation               (Graph)
  2. Show Precedent Graph             (Graph)
  3. Most Influential Judgment         (Graph Analysis)
  4. Detect Circular Citations         (Cycle Detection)
  5. Go Back
  Choose option: 2

--- Legal Precedent Graph ---
Case 104 cites -> 103 101 102
Case 105 cites -> 103 101
Case 107 cites -> 103
Case 108 cites -> 104 102
Case 109 cites -> 104
Case 110 cites -> 130
Case 113 cites -> 103
Case 114 cites -> 103 104 102
Case 115 cites -> 101
Case 116 cites -> 104 108
Case 120 cites -> 121
Case 121 cites -> 122
Case 122 cites -> 120

--- Legal Analytics ---
  1. Add Legal Citation               (Graph)
  2. Show Precedent Graph             (Graph)
  3. Most Influential Judgment         (Graph Analysis)
  4. Detect Circular Citations         (Cycle Detection)
  5. Go Back
  Choose option: 3
```

```
pratikswain@Mac DSA final project % cd "/Users/pratikswain/Desktop/DSA final project/" && g++ nat
ionalcourt.cpp -o nationalcourt && "/Users/pratikswain/Desktop/DSA final project/"nationalcourt
  4. Detect Circular Citations         (Cycle Detection)
  5. Go Back
  Choose option: 3

--- Most Influential Judgment ---
Case ID: 103 | Cited 5 time(s)
Title: Bail Application: State vs. Khan

--- Legal Analytics ---
  1. Add Legal Citation              (Graph)
  2. Show Precedent Graph            (Graph)
  3. Most Influential Judgment        (Graph Analysis)
  4. Detect Circular Citations        (Cycle Detection)
  5. Go Back
  Choose option: 4

[WARNING] Circular citation detected! Review precedent links.

--- Legal Analytics ---
  1. Add Legal Citation              (Graph)
  2. Show Precedent Graph            (Graph)
  3. Most Influential Judgment        (Graph Analysis)
  4. Detect Circular Citations        (Cycle Detection)
  5. Go Back
  Choose option: 5

========== National Judicial Case Management System ==========
  1. CASE MANAGEMENT
  2. LEGAL ANALYTICS
  3. APPEAL TRACKING
  4. DASHBOARD
  5. Exit
============================================================
  Choose option: 3

--- Appeal Tracking ---
  1. Add Appeal Link                 (Graph)
  2. Track Appeal Journey            (DFS)
  3. Go Back
  Choose option: 1

Enter Lower Court Case ID: 56
Enter Higher Court Case ID: 78
Appeal linked: 56 -> 78

--- Appeal Tracking ---
  1. Add Appeal Link                 (Graph)
  2. Track Appeal Journey            (DFS)
  3. Go Back
  Choose option: 2

Enter Starting Case ID: 56

Appeal Journey: 56 -> 78 -> END

--- Appeal Tracking ---
  1. Add Appeal Link                 (Graph)
  2. Track Appeal Journey            (DFS)
  3. Go Back
  Choose option: 2

Enter Starting Case ID: 101

Appeal Journey: 101 -> 201 -> 301 -> 401 -> END
```



Results and Findings

The developed system successfully demonstrates the practical implementation of Data Structures and Algorithms in a real-world judicial environment.

Findings

- Case registration and retrieval operate efficiently.
- Binary Search significantly improves search speed.
- Heap-based prioritization effectively handles urgent cases.
- Graph structures successfully model legal precedent relationships.
- DFS efficiently traces appeal paths.
- Circular citation detection helps identify inconsistent legal references.
- Citation analysis identifies landmark judgments based on citation frequency.

Sample Dataset Statistics

The system loads:

- 20 judicial cases
- Criminal, Civil, Constitutional, and Commercial categories
- Multiple citation relationships
- Multi-level appeal hierarchies
- Circular citation test cases for validation

Conclusion

The National Judicial Case Management System successfully integrates multiple Data Structures and Algorithms to simulate real-world court operations.

The project demonstrates the use of Arrays, Queues, Heaps, Binary Search, Graphs, DFS, and Cycle Detection in solving practical legal management challenges. The system provides efficient case registration, priority handling, legal precedent analysis, and appeal tracking while maintaining scalability and simplicity.

This project highlights how fundamental DSA concepts can be applied to build intelligent and efficient software systems for judicial administration.

GitHub Repository Link

GitHub
Repository:-https://github.com/pratikswain070-blip/National-Judicial-Case-Management-System